MMOT225: Bulut Bilişim
Hafta 6: Bulut Hizmet (SPI) ve Dağıtım Modelleri

Dr. Öğr. Üyesi Hasan Oğuz

İstanbul Okan Üniversitesi | Meslek Yüksekokulu

Bugün Neler Öğreneceğiz?

Geleneksel on-premise (yerinde) sistemlerden buluta geçişte kontrol ve sorumlulukların nasıl değiştiğini inceleyeceğiz.

- **Sorumluluk Modeli (Shared Responsibility):** Pizzacı Analojisi.
- **SPI Modeli (Hizmet Türleri):**
  - IaaS (Infrastructure as a Service - Hizmet olarak Altyapı)
  - PaaS (Platform as a Service - Hizmet olarak Platform)
  - SaaS (Software as a Service - Hizmet olarak Yazılım)
- **Bulut Dağıtım Modelleri:** Public, Private, Hybrid ve Multi-Cloud.

Hizmet Modellerini Anlamak: "Pizza as a Service" Analojisi

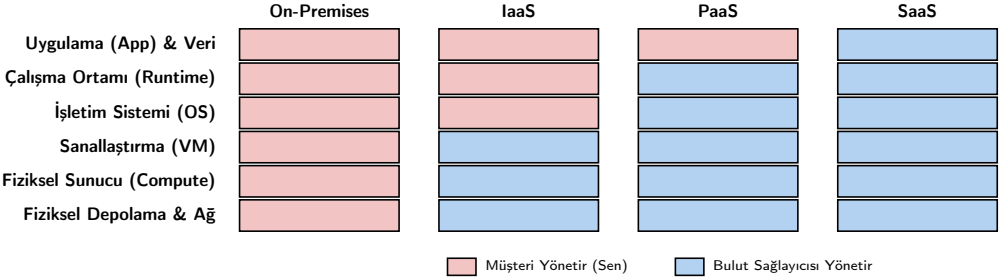
Bulut bilişimi anlamanın en iyi yolu, evde pizza yapmak ile dışarıdan pizza söylemek arasındaki farkı düşünmektir.

Bileşenler	Evde Yapım (On-Premise)	Marketten Dondurulmuş (IaaS)	Pizza Teslimatı (PaaS)	Restoranda Yeme (SaaS)
Yemek Masası & İçecek	Senin	Senin	Senin	Restoranın
Fırın & Gaz/Elektrik	Senin	Senin	Pizzacının	Restoranın
Pizza Hamuru & Malzeme	Senin	Marketin	Pizzacının	Restoranın

Kırmızı (Senin): Yönetim ve bakım **senin** sorumluluğunda.
Mavi (Servis Sağlayıcı): Yönetim ve bakım **bulut sağlayıcısının** sorumluluğunda.
Buluta (sağa) doğru gittikçe kontrolün azalır, ancak rahatlığın ve hızın artar.

Bilişim Katmanları (IT Stack) Üzerinden Karşılaştırma

Şimdi Pizza örneğini **Gerçek IT Dünyasına** uyarlayalım. Hangi katmanı kim yönetiyor?



1. IaaS (Infrastructure as a Service) - Konsept ve Temel Mantık

IaaS, bulut bilişimin **temel yapı taşıdır**. Fiziksel bir sunucu satın almak yerine; işlem gücünü (CPU), belleği (RAM), depolamayı ve ağ altyapısını internet üzerinden saatlik/dakikalık olarak kiralarsınız.

Temel Özellikleri

- **Donanım Bağımsızlığı:** Alt katmandaki fiziksel sunucunun arızalanması sizi etkilemez; sağlayıcı VM'nizi başka sunucuya taşır.
- **Ölçeklenebilirlik:** Ram yetmediğinde sunucu kapatıp saniyeler içinde RAM miktarını artırarak tekrar açabilirsiniz.
- **OpEx Modeli:** İlk yatırım maliyeti yoktur, sadece cihaz açık kaldığı sürece fatura ödersiniz.

Laboratuvar Bağlantısı

Geçen haftalarda VirtualBox üzerinde oluşturduğumuz **"Sanal Makine (VM)"** işleminin tam olarak buluttaki karşılığıdır. Tek fark, hipervizörü bizim değil, bulut firmasının yönetmesidir.

1. IaaS (Infrastructure as a Service) - Kontrol ve Sorumluluk

IaaS modeli, SPI (SaaS/PaaS/IaaS) piramidinde **müşteriye en yüksek kontrolü** veren, ancak karşılığında en çok sorumluluğu yükleyen modeldir.

Bulut Sağlayıcısının Sorumluluğu

- Veri merkezinin fiziksel güvenliği, elektrik ve soğutma altyapısı.
- Fiziksel sunucuların ve ağ cihazlarının bakımı.
- Hipervizör yazılımının (Sanallaştırma katmanı) sorunsuz çalışması.

Sağlayıcı "Donanım çalışıyor, gerisi bende değil" der.

Sizin (Müşterinin) Sorumluluğunuz

- **İşletim Sistemi:** İşletim sistemini seçmek, kurmak ve güncellemelerini (Patch) yapmak.
- **Güvenlik:** Güvenlik duvarı (Firewall) kuralları ve port izinleri.
- **Yazılım:** İçine kurulacak Docker, Veritabanı veya Web sunucusunun yönetimi.

1. IaaS (Infrastructure as a Service) - Kullanım Senaryoları

Neden bir şirket PaaS veya SaaS yerine her şeyi kendisinin yönetmesi gereken IaaS modelini seçer?

İdeal Kullanım Senaryoları

- **Lift and Shift (Kaldır ve Taşı):** Şirketteki eski sistemleri kodunu hiç değiştirmeden doğrudan buluta taşımak.
- **Test ve Geliştirme Ortamları:** Sadece mesai saatlerinde çalışıp akşam kapatılarak maliyet tasarrufu sağlanan test sunucuları.
- **Özel Güvenlik/Regülasyon:** İşletim sisteminin her dosyasına **Root/Admin** seviyesinde müdahale edilmesi gereken kritik görevler.

Sektördeki Lider IaaS Ürünleri

Dev şirketler, IaaS (Sanal Sunucu) hizmetlerine kendi marka isimlerini verirler:

- **AWS:** EC2 (Elastic Compute Cloud)
- **Microsoft Azure:** Azure Virtual Machines
- **Google Cloud (GCP):** Compute Engine
- **DigitalOcean:** Droplets

2. PaaS (Platform as a Service) - Konsept ve Temel Mantık

PaaS, **yazılım geliştiriciler (Developers)** için tasarlanmış bir hizmet modelidir. Sunucu satın almak veya işletim sistemi kurmakla uğraşmazsınız; size doğrudan kodunuzu çalıştırabileceğiniz hazır bir **Platform (Çalışma Ortamı)** verilir.

Temel Özellikleri

- **Altyapı Soyutlaması:** Geliştirici, arka planda kaç RAM veya hangi işlemci çalıştığını bilmez ve ilgilenmez.
- **Hızlı Dağıtım:** Kodu yazarsınız, sağlayıcıya "Al bunu çalıştır" dersiniz. Geri kalan her şey otomatiktir.
- **Dahili Araçlar:** Veritabanı bağlantıları, loglama ve performans izleme araçları platforma entegredir.

Laboratuvar Bağlantısı

Bir önceki labda kendi web sitemizi (`index.html`) yayınlamak için Ubuntu kurup, içine Docker yükleyip Nginx ayağa kaldırmakla uğraşmıştık (IaaS yaklaşımı). **PaaS** kullansaydık, sadece `index.html` dosyamızı platforma (Örn: Vercel, Netlify) yüklememiz yeterli olacaktı!

2. PaaS (Platform as a Service) - Kontrol ve Sorumluluk

PaaS modelinde kontrolün büyük bir kısmı bulut firmasına geçer. Müşteri (Siz) sadece uygulamanın kalitesinden ve içindeki veriden sorumlu olursunuz.

Bulut Sağlayıcısının Sorumluluğu

- Fiziksel donanım, ağ ve sanallaştırma.
- **İşletim Sistemi:** Güvenlik yamaları, güncellemeler ve lisanslamalar.
- **Çalışma Ortamı (Runtime):** Python, Node.js veya Java sürümlerinin güncel ve sorunsuz çalışması.

Sağlayıcı: "Sunucuyu ben yönetirim, sen koduna odaklan."

Sizin (Müşterinin) Sorumluluğunuz

- **Uygulama Kodu:** Yazdığınız kodun hatasız çalışması ve algoritmik verimliliği.
- **Veri Yönetimi:** Veritabanındaki müşteri bilgilerinin güvenliği ve yedeklenme politikaları.
- **Uygulama Erişimi:** Kimlerin bu uygulamaya giriş yapabileceği (Yetkilendirme).

2. PaaS (Platform as a Service) - Senaryolar ve Endüstri Liderleri

PaaS, özellikle sistem yöneticisi (SysAdmin) barındırmayan çevik yazılım ekiplerinin en çok tercih ettiği modeldir.

İdeal Kullanım Senaryoları

- **Hızlı Prototipleme (Time-to-Market):** Yeni bir girişimin (Startup) uygulamasını en kısa sürede pazara sunmak istemesi.
- **Yönetilen Veritabanları:** Şirketlerin veritabanı kurmak yerine, yedeklemesi otomatik yapılan hazır veritabanı (DBaaS) kiralaması.
- **Mikroservis Mimarileri:** Birbirinden bağımsız küçük API'lerin hızlıca yayınlanması.

Sektördeki Lider PaaS Ürünleri

Sadece kodunuzu atıp arkanıza yaslandığınız servisler:

- **Heroku:** Geliştiricilerin en sevdiği klasik platform.
- **AWS Elastic Beanstalk**
- **Google App Engine**
- **Vercel / Netlify:** Modern Frontend (Web arayüzü) yayınlama platformları.

3. SaaS (Software as a Service) - Konsept ve Temel Mantık

SaaS, **son kullanıcılar** (bizler ve şirket çalışanları) için tasarlanmış en üst katman hizmet modelidir. Yazılımı bilgisayarınıza kurmazsınız; tamamen internet üzerinden (genellikle bir web tarayıcısı ile) erişim sağlarsınız.

Temel Özellikleri

- **Kurulum Yok:** CD, USB veya indirme işlemleriyle uğraşılmaz. Hesabını aç ve hemen kullanmaya başla.
- **Abonelik Modeli:** Lisans satın almak yerine aylık/yıllık kiralama (Pay-as-you-go) yapılır.
- **Çoklu Kiracı (Multi-tenant):** Milyonlarca kullanıcı aynı yazılım altyapısını birbirinden izole olarak kullanır.

Merkezi Güncelleme

Geleneksel yazılımlarda (Örn: eski Microsoft Office) güncelleme için her bilgisayara yama yüklemek gerekirken; SaaS modelinde sağlayıcı yazılımı merkezden günceller ve ertesi gün sisteme giren herkes yeni versiyonu kullanmaya başlar.

3. SaaS (Software as a Service) - Kontrol ve Sorumluluk

SaaS modeli, SPI piramidinde müşteriye **en az kontrolü** veren, ancak karşılığında tüm teknik yükü ve sorumluluğu bulut sağlayıcısına devreden modeldir.

Bulut Sağlayıcısının Sorumluluğu

- Donanım, ağ, sanallaştırma ve işletim sistemi.
- Yazılımın kodlanması, veritabanı bakımı ve performans optimizasyonları.
- **Yüksek Erişilebilirlik (HA):** Sistemin 7/24 kesintisiz çalışması ve felaket anında kurtarılması.

Sağlayıcı: "Her şeyi ben yönetirim, sen sadece kullan."

Sizin (Müşterinin) Sorumluluğunuz

- **Veri Sahipliği:** Sisteme yüklediğiniz belgeler, müşteri listeleri veya kişisel verilerinizin yönetimi.
- **Erişim Yönetimi:** Şirket içinden kimlerin, hangi yetkilerle bu sisteme gireceği (Yetkilendirme ve Şifre güvenliği).
- **Cihaz Güvenliği:** Yazılıma bağlandığınız kendi bilgisayarınızın veya telefonunuzun güvenli olması.

3. SaaS (Software as a Service) - Senaryolar ve Endüstri Liderleri

Günümüzde hem bireysel hayatımızın hem de kurumsal işleyişin neredeyse tamamı SaaS ürünleri üzerinden yürümektedir.

İdeal Kullanım Senaryoları

- **Ortak İhtiyaçlar:** E-posta, ofis programları, muhasebe gibi her şirketin standart olarak ihtiyaç duyduğu uygulamalar.
- **Geçici İhtiyaçlar:** Sadece birkaç ay kullanılacak bir proje yönetim aracı için sıfırdan sunucu kurmak yerine buluttan kiralamak.
- **Mobilite:** Çalışanların ofis dışında, telefon veya tabletlerinden iş verilerine anında erişebilmesi.

Sektördeki Lider SaaS Ürünleri

Lisansını satın almadığımız, aylık/yıllık kiraladığımız günlük uygulamalar:

- **İş ve İletişim:** Google Workspace, Microsoft 365, Zoom, Slack
- **Müşteri Yönetimi (CRM):** Salesforce
- **Depolama:** Google Drive, Dropbox
- **Eğlence:** Netflix, Spotify

Bulut Dağıtım Modelleri: 1. Genel Bulut (Public Cloud)

SPI modeli "Ne tür bir hizmet alıyoruz?" sorusunu yanıtlarken, Dağıtım modeli **"Bu altyapı kimin için inşa edildi ve nerede duruyor?"** sorusunu yanıtlar.

Genel Bulut Özellikleri

- **Çoklu Kiracı (Multi-tenant):** AWS, Azure veya GCP'ye ait devasa veri merkezlerindeki donanımlar, birbirinden izole edilmiş milyonlarca müşteri tarafından ortak kullanılır.
- **Maliyet Avantajı:** İlk donanım yatırımı (CapEx) sıfırdır. Tamamen "kullandığın kadar öde" (OpEx) modeli geçerlidir.

Kullanım Senaryoları

- **Ölçeklenebilirlik:** Trafiği tahmin edilemeyen (örn: e-ticaret siteleri, oyun sunucuları) uygulamalar.
- **Girişimler (Startups):** Kendi veri merkezini kuracak sermayesi olmayan, hızlıca pazara girmek isteyen teknoloji şirketleri.

Bulut Dağıtım Modelleri: 2. Özel Bulut (Private Cloud)

Özel bulut, altyapının **sadece tek bir şirkete veya kuruma özel** olarak tahsis edildiği modeldir. Genellikle kurumun kendi fiziksel veri merkezinde (On-Premise) barındırılır, ancak bulutun esneklik (self-servis) özelliklerini taşır.

Özel Bulut Özellikleri

- **Tek Kiracı (Single-tenant):** Donanım ve ağ kaynakları dışarıdaki hiçbir şirketle paylaşılmaz.
- **Tam Kontrol:** Donanım konfigürasyonundan ağ topolojisine kadar her şey kurumun IT ekibinin kontrolündedir. Yüksek donanım ve personel maliyeti gerektirir.

Kullanım Senaryoları

- **Sıkı Regülasyonlar:** Yasal zorunluluklar gereği verilerini ülke dışına veya üçüncü şahıs sunucularına çıkaramayan bankalar, hastaneler ve kamu kurumları.
- **Gecikme (Latency) Hassasiyeti:** Üretim bandındaki cihazlarla milisaniyenin altında haberleşmesi gereken endüstriyel sistemler.

Bulut Dağıtım Modelleri: 3. Hibrit ve Çoklu Bulut (Multi-Cloud)

Günümüzün kurumsal dünyasında şirketler tek bir modele bağlı kalmak yerine karma stratejiler (pragmatik yaklaşım) benimserler.

Hibrit Bulut (Hybrid Cloud)

- **Public + Private** kombinasyonudur.
- **Mantık:** Şirket, gizli müşteri veritabanını kendi veri merkezinde (Private) tutarken; herkese açık web sitesi arayüzünü AWS'de (Public) barındırır.
- İki ortam birbirine **güvenli VPN tünelleri** (Örn: AWS Direct Connect) ile bağlanır. En popüler kurumsal modeldir.

Çoklu Bulut (Multi-Cloud) Stratejisi

- **Vendor Lock-in (Sağlayıcıya Bağımlılık)** riskini azaltmak için birden fazla Public Cloud (Aynı anda AWS, Azure ve GCP) kullanılmasıdır.
- **Mantık:** Yapay zeka servisleri için Google Cloud'u kullanırken, Windows sunucuları için Azure'u tercih edip hepsini tek bir ağda birleştirmek.

Adım 1: Pardus (Debian) Üzerine Docker Kurulumu

Bu laboratuvarda yerli işletim sistemimiz **Pardus** (Debian tabanlı) üzerinde çalışıyoruz. Pardus'un paket yöneticisi (`apt`), Ubuntu ile aynı mantıkta çalışır.

Terminal Kurulum Komutları

```
$ sudo apt update
$ sudo apt install docker.io -y
```

Servisi Başlatma ve Aktif Etme

Docker'ın her sistem açılışında otomatik başlaması için:

```
$ sudo systemctl enable --now docker
$ sudo docker --version
```

Adım 2: Kendi Uygulamamızı Yazalım (Python Hesap Makinesi)

Hazır imajları kullanmak yerine **kendi mikroservisimizi** geliştireceğiz. Web tabanlı basit bir toplama işlemi yapan Python uygulaması yazalım.

1. Proje Klasörünü Oluşturma

```
$ mkdir ~/hesap-makinesi && cd ~/hesap-makinesi
$ nano app.py
```

2. app.py İçeriği (Nano içine yapıştırıp kaydedin)

```
from flask import Flask
app = Flask(__name__)

@app.route('/topla/<a>/<b>')
def topla(a, b):
    sonuc = int(a) + int(b)
    return f"<h1>Hesap Makinesi: {a} + {b} = {sonuc}</h1>"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Adım 3: Reçeteyi Yazmak (Dockerfile)

Docker'a bu Python kodunu nasıl çalıştıracağını söylememiz gerekir. Bunun için `Dockerfile` adında uzantısız bir reçete (talimat) dosyası oluştururuz.

1. Dockerfile Oluşturma

```
$ nano Dockerfile
```

2. `Dockerfile` İçeriği

```
FROM python:3.9-slim # Alt yapi olarak hafif bir Python Linux imaji al
WORKDIR /app # Konteyner icinde /app klasorune gec
RUN pip install flask # Web sunucusu icin gerekli Flask kutuphanesini kur
COPY app.py . # Bizim yazdigimiz app.py dosyasini konteynere kopyala
CMD ["python", "app.py"] # Konteyner calistiginda bu komutu tetikle
```

Adım 4: İmajı Derleme (Build) ve Çalıştırma (Run)

Reçetemiz hazır. Şimdi bu reçeteden kendimize ait bir **Docker İmajı** üreteceğiz (Tıpkı yazılımı derler (compile) gibi).

1. İmajı Üretme (Build)

```
$ sudo docker build -t benim-hesap-makinem .
```

Not: Sondaki nokta (.) işaretini unutmayın. "Dockerfile'ı bulunduğum dizinde ara" demektir.

2. Kendi Uygulamamızı Ayağa Kaldırma (Run)

Uygulamamız 5000 portunda çalışacak şekilde ayarlandı. Dış dünyadaki 8080 portunu içeriye yönlendirelim:

```
$ sudo docker run -d -p 8080:5000 --name python-app benim-hesap-makinem
$ sudo docker ps
```

Adım 5: Uzaktan Bağlantı ve Test (PaaS Deneyimi)

Artık yazdığınız kod, Pardus sanal makinesi üzerinde izole bir konteynerde çalışıyor. Şimdi asıl fiziksel makinenizden (Windows/Mac) bu servise bağlanalım.

1. Pardus'un IP'sini Öğrenin

```
$ ip a
```

Örnek çıktıdan IP'nizi (Örn: 192.168.1.100) not alın.

2. Fiziksel Makineden İstek Atma

Windows/Mac bilgisayarınızda bir tarayıcı açın ve şu adresi girin (A ve B yerine istediğiniz sayıları yazın):

```
http://<PARDUS-IP>:8080/topla/15/25
```

Bulut Perspektifi: Yazdığınız kodu çalıştıracak sunucu ortamını (Python versiyonu, bağımlılıklar) `Dockerfile` ile standartlaştırdınız. Bunu AWS'ye de gönderseneniz **birebir aynı şekilde** çalışacaktır (PaaS).

Adım 1: Yeni Senaryo - IoT Sensör Simülasyonu (Background Worker)

Önceki örneğimizde bir Web Sunucusu (Flask) ayağa kaldırdık ve biz istek attıkça cevap verdi. Bulut mimarisinde bir de kendi kendine sürekli çalışan **Arka Plan İşçileri (Background Workers)** vardır.

Bu laboratuvarda, durmadan rastgele sıcaklık verisi üreten bir IoT sensör yazılımını Python ile kodlayıp Docker içine gömeceğiz.

Proje Klasörünü Oluşturma

```
$ mkdir ~/iot-sensor && cd ~/iot-sensor
$ nano sensor.py
```

Not: Tüm konteynerlerin internete açık portları (Web API) olmak zorunda değildir. Bazıları sadece arka planda veri işler veya veritabanına kayıt atar.

Adım 2: Python Kodunu Yazalım (Ortam Değişkenleri ile)

Kritik Bulut Kuralı: Bulutta kodun içine sabit değer (Hardcode) yazılmaz. Ayarlar, **Ortam Değişkenleri (Environment Variables)** ile dışarıdan enjekte edilir.

sensor.py İçeriği (Nano içine yapıştırıp kaydedin)

```
import time
import random
import os

# Ayarlari isletim sisteminden (Docker'dan) cekiyoruz
sensor_adi = os.getenv("SENSOR_ADI", "Varsayilan_Cihaz")
bekleme_suresi = int(os.getenv("BEKLEME", "3"))

print(f"--- {sensor_adi} Baslatildi. Her {bekleme_suresi} saniyede veri gonderecek ---", flush=True)

while True:
    sicaklik = random.uniform(20.0, 35.0)
    print(f"[{sensor_adi}] Anlik Sicaklik: {sicaklik:.2f} C", flush=True)
    time.sleep(bekleme_suresi)
```

Adım 3: İşçi Konteyneri İçin Dockerfile Reçetesi

Bu sefer dışarıya açacağımız bir port (`EXPOSE`) yok. Sadece Python'un çalışmasını sağlayacağız.

1. Dockerfile Oluşturma

```
$ nano Dockerfile
```

2. Dockerfile İçeriği

```
FROM python:3.9-slim # Hafif Python imajini kullan
WORKDIR /app # Calisma dizinini ayarla
COPY sensor.py . # Kodumuzu konteynerin icine kopyala
CMD ["python", "sensor.py"] # Konteyner basladiginda kodu tetikle
```

Adım 4: İmajı Derleme ve Parametrik Çalıştırma (-e)

Reçetemizi derleyelim ve **aynı imajdan**, farklı ayarlara sahip **iki farklı konteyner** ayağa kaldıralım. Ortam değişkenlerini -e parametresi ile enjekte edeceğiz.

1. İmajı Derleme (Build)

```
$ sudo docker build -t python-sensor .
```

2. Konteynerleri Başlatma (-e Parametresi)

```
# 1. Konteyner: Mutfak Sensoru (Saniyede 1 veri)
$ sudo docker run -d --name mutfak_sensoru -e SENSOR_ADI="Mutfak_Sicaklik" -e BEKLEM

# 2. Konteyner: Depo Sensoru (5 saniyede 1 veri)
$ sudo docker run -d --name depo_sensoru -e SENSOR_ADI="Depo_Sicaklik" -e BEKLEME="5
```

Adım 5: Arka Plan İşlemlerini Canlı İzleme (Logs -f)

Konteynerlerimiz `-d` (detach) parametresiyle arka planda çalışmaya başladı. Peki ürettikleri veriyi nasıl göreceğiz?

Canlı Log Takibi (`-f` Parametresi)

Aşağıdaki komut, konteynerin terminal çıktısına canlı olarak bağlanır. Çıkmak için **Ctrl + C** tuşlarına basın.

```
$ sudo docker logs -f mutfak_sensoru
```

Bulut Mimarisi Perspektifi

Şu an tek bir `sensor.py` dosyasından **iki farklı cihaz** ürettik. SaaS uygulamalarında (Örneğin Netflix'te) video işleyen arka plan sunucuları tam olarak bu şekilde, aynı imaja farklı **ENV (Ortam Değişkeni)** verilerek binlerce kopya halinde çoğaltılır.

Sorular?